

MACHINE LEARNING CHO LẬP TRÌNH VIÊN

Kiến Thức Cơ Bản Đến Trung Cấp

1. Machine Learning là gì?

Machine Learning (ML) là một nhánh của Artificial Intelligence (AI), giúp máy tính học từ dữ liệu và tự cải thiện mà không cần được lập trình một cách tường minh.

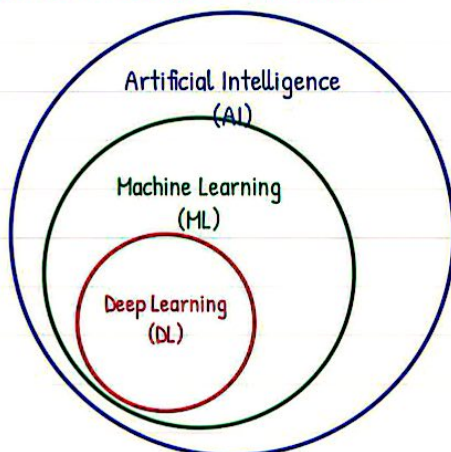
Nói một cách đơn giản:

"Học từ dữ liệu trong quá khứ để đưa ra quyết định hoặc dự đoán tốt hơn cho tương lai."

2. Ví Dụ Thực Tế

- **Netflix** gợi ý phim dựa trên lịch sử xem của bạn.
- **Gmail** tự động lọc email spam.
- **Amazon** đề xuất sản phẩm bạn có thể thích.
- **Google Maps** dự đoán tình trạng giao thông và chỉ đường tốt nhất.

3. AI → ML → Deep Learning



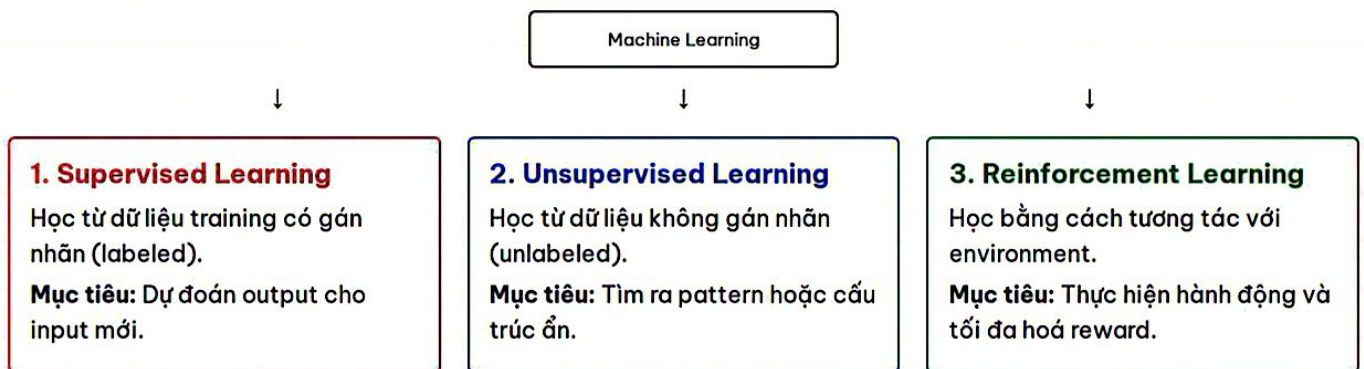
- AI là khái niệm rộng hơn về việc làm cho máy móc trở nên thông minh.
- ML là một tập con của AI, giúp máy móc học từ dữ liệu.
- Deep Learning là một tập con của ML, sử dụng Neural Network với nhiều lớp.

4. Lập Trình Truyền Thống so với Machine Learning

Lập Trình Truyền Thống	Machine Learning
• Bạn viết ra các quy tắc tường minh. • Máy tính làm theo các quy tắc đó. • Ví dụ: nếu age > 18 → Eligible • Phù hợp với bài toán đơn giản, rõ ràng.	• Bạn cung cấp dữ liệu. • Máy tính học các pattern từ dữ liệu. • Ví dụ: học pattern về điều kiện đủ tuổi từ dữ liệu quá khứ. • Phù hợp với bài toán phức tạp, thực tế.

2. Các Loại Machine Learning

Machine Learning được chia thành **ba loại chính**, dựa trên loại dữ liệu và cách tiếp cận học.



Chi Tiết Kèm Ví Dụ

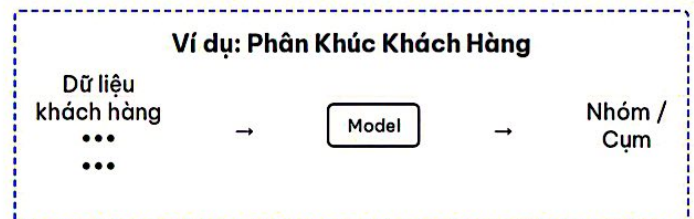
① Supervised Learning

- Sử dụng dữ liệu có nhãn (input + output).
- Model học ánh xạ (mapping) từ input sang output.
- Các loại: **Regression, Classification**



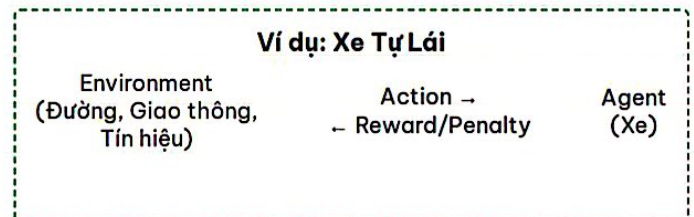
② Unsupervised Learning

- Sử dụng dữ liệu không nhãn (chỉ có input).
- Model tìm ra pattern hoặc nhóm dữ liệu.
- Các loại: **Clustering, Dimensionality Reduction, Association**



③ Reinforcement Learning

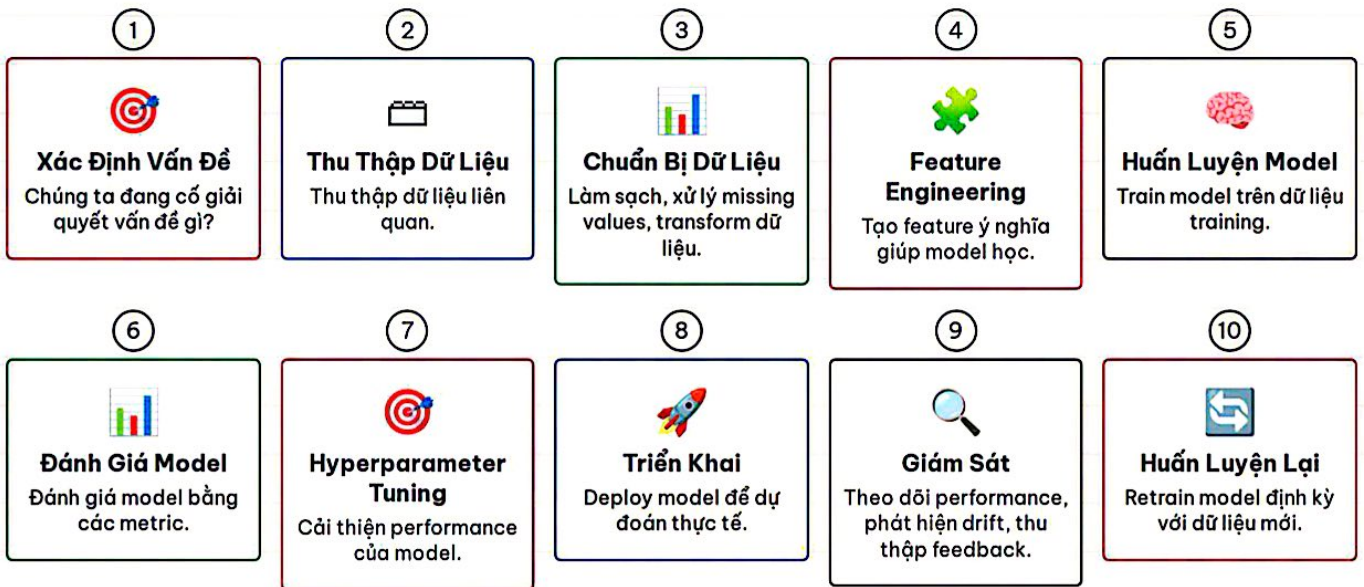
- Agent học qua thử và sai (trial & error).
- Nhận reward khi hành động tốt, phạt khi hành động xấu.
- Mục tiêu:** Tối đa hoá tổng reward.



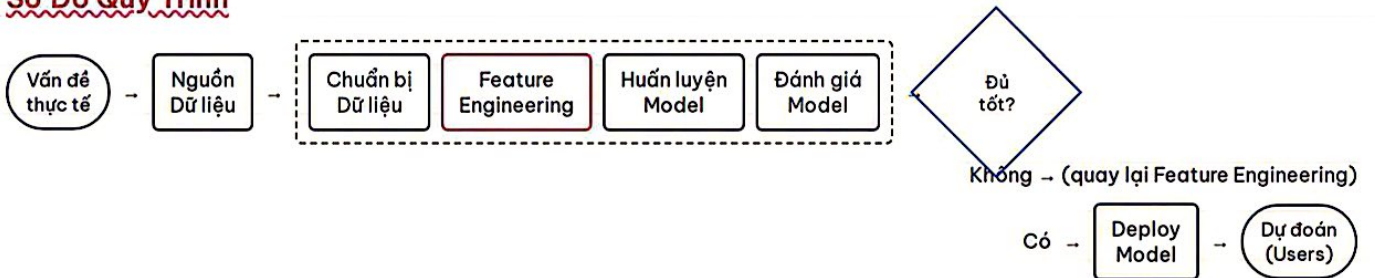
Chọn đúng loại ML dựa trên bài toán và dữ liệu bạn có.

3. Quy Trình Machine Learning

Machine Learning tuân theo một quy trình từng bước để đi từ vấn đề đến dự đoán và cải tiến.



Sơ Đồ Quy Trình



Điểm Quan Trọng

- Không phải bài toán nào cũng đi theo đường thẳng.
- Bạn có thể lặp lại giữa các bước để cải thiện kết quả.
- Dữ liệu tốt + feature tốt = model tốt hơn.
- Giám sát và huấn luyện lại giúp model chính xác theo thời gian.

Tóm Tắt Quy Trình

Vấn đề → Dữ liệu → Chuẩn bị → Feature
→ Huấn luyện → Đánh giá → Tuning → Deploy
→ Giám sát → Huấn luyện lại

4. Dataset, Feature & Label

Trong Machine Learning, chúng ta làm việc với dữ liệu.

Hiểu về **dataset**, **feature** và **label** là bước đầu tiên để xây dựng bất kỳ model ML nào.

→ 1. Dataset

Dataset là một tập hợp các example hoặc record.

Mỗi record chứa thông tin về một đối tượng.

Ví dụ: Dataset Giá Nhà

House ID	Diện tích (sq.ft)	Phòng ngủ	Khu vực	Giá (₹)
101	1200	2	Mumbai	50.000.000
102	1500	3	Delhi	65.000.000
103	1800	3	Bangalore	80.000.000
104	2000	4	Mumbai	95.000.000
105	2500	4	Delhi	110.000.000

Mỗi hàng → một example
(một căn nhà)

Mỗi cột → thông tin
về căn nhà

→ 2. Feature

Feature (hay biến đầu vào) là các cột mô tả đặc điểm của dữ liệu.

Chúng được model sử dụng để học pattern.

Trong ví dụ trên, feature gồm:

- Diện tích (sq.ft)
- Phòng ngủ
- Khu vực

Những feature này giúp model hiểu căn nhà và dự đoán giá.

→ 3. Label (Target)

Label (hay biến mục tiêu) là thứ chúng ta muốn model dự đoán.

Đó chính là output.

Trong ví dụ trên, label là:

- Giá (₹)

Model học từ feature và cố dự đoán label (giá nhà).

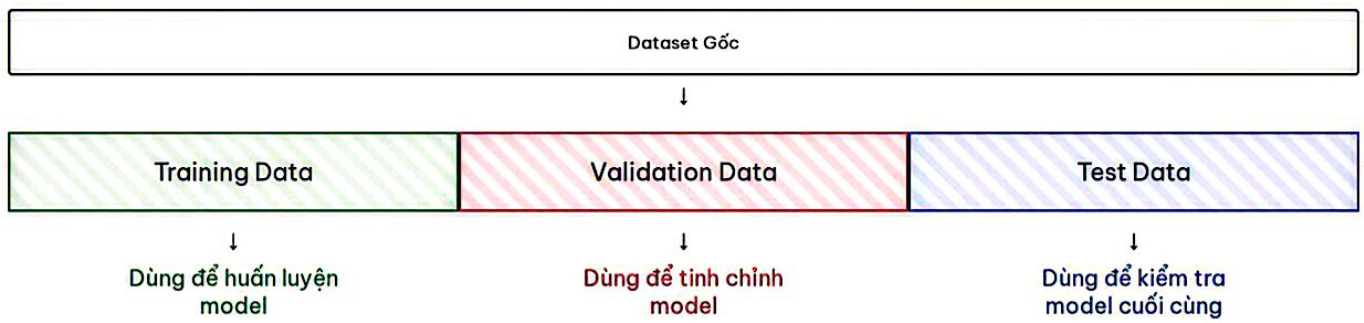
Tóm lại:

- **Dataset** → Toàn bộ dữ liệu
- **Feature** → Input (biến độc lập)
- **Label** → Output (biến mục tiêu)

Model dùng feature để dự đoán label.

5. Training, Validation & Test Data

Để xây dựng một model Machine Learning tốt, chúng ta chia dataset thành **ba phần**.



1 Training Data

- Phần lớn nhất của dữ liệu.
- Dùng để huấn luyện model.
- Model học pattern từ dữ liệu này.

Mục đích:
Giúp model học.

2 Validation Data

- Phần nhỏ hơn của dữ liệu.
- Dùng để tune hyperparameter và cải thiện model.
- Giúp chọn ra model tốt nhất.

Mục đích:
Giúp tuning và tránh overfitting.

3 Test Data

- Dữ liệu hoàn toàn chưa từng thấy đối với model.
- Chỉ dùng một lần duy nhất ở cuối.
- Cho đánh giá khách quan về hiệu năng model cuối cùng.

Mục đích:
Kiểm tra model hoạt động thế nào trong thực tế (dữ liệu chưa thấy).

Tỷ Lệ Chia Thường Gặp (Ví Dụ)

Training Data 70%	Validation Data 15%	Test Data 15%
----------------------	------------------------	------------------

Lưu ý:

- Tỷ lệ chính xác có thể thay đổi (ví dụ: 80-10-10 hoặc 70-15-15).
- Điều quan trọng là giữ test data hoàn toàn tách biệt và chưa từng thấy.

6. Data Preprocessing

Dữ liệu thô thường lộn xộn và chưa phù hợp để đưa vào model ML.

Data preprocessing giúp **làm sạch và biến đổi dữ liệu** để sử dụng cho việc huấn luyện.

Tại sao lại quan trọng?

- Cải thiện độ chính xác của model
- Tăng tốc quá trình huấn luyện
- Giúp xử lý dữ liệu thiếu và không nhất quán
- Giúp dữ liệu dễ hiểu hơn

Các Bước Preprocessing Thường Gặp

① Xử Lý Missing Values

- Điền giá trị thiếu hoặc xóa row/column có quá nhiều missing values.

Trước			→	Sau		
Age	Income	City		Age	Income	City
25	50000	Delhi		25	50000	Delhi
-	60000	Mumbai		28	60000	Mumbai
30	-	Pune		30	55000	Pune

(Dùng Mean/Median để điền)

② Xoá Dữ Liệu Trùng Lặp

- Xoá các row trùng lặp để tránh gây bias.

Trước		→	Sau	
A, 25, Delhi			A, 25, Delhi	
A, 25, Delhi			B, 30, Mumbai	
B, 30, Mumbai			C, 22, Pune	
C, 22, Pune				

③ Encode Dữ Liệu Categorical

- Chuyển category thành số để model có thể hiểu được.

Ví dụ:
City → Delhi, Mumbai, Pune → City → 0, 1, 2
(Label Encoding / One-Hot Encoding)

④ Feature Scaling

- Đưa tất cả feature về cùng một scale.
- Phương pháp phổ biến: **Standardization** (Z-score), **Normalization** (0 đến 1).

Trước Khi Scale		→	Sau Khi Scale (0 đến 1)	
Age: 10 - 70			Age: 0.0 - 1.0	
Income: 20.000 - 200.000			Income: 0.0 - 1.0	

⑤ Phát Hiện & Xử Lý Outlier

- Phát hiện giá trị bất thường có thể ảnh hưởng xấu đến model.
- Xử lý bằng cách xoá, giới hạn (capping) hoặc transform.

Ví dụ (Income)
20k - 30k - 40k - 50k - 60k --- **5.000.000**
5.000.000 là outlier → xoá hoặc cap lại.

⑥ Feature Engineering

- Tạo feature mới từ dữ liệu hiện có.
- Giúp cải thiện độ chính xác của model.

Ví dụ:

Date of Birth → Age

Total = Quantity × Price → Total Amount

Tóm lại: Dữ liệu sạch → Học tốt hơn → Dự đoán tốt hơn

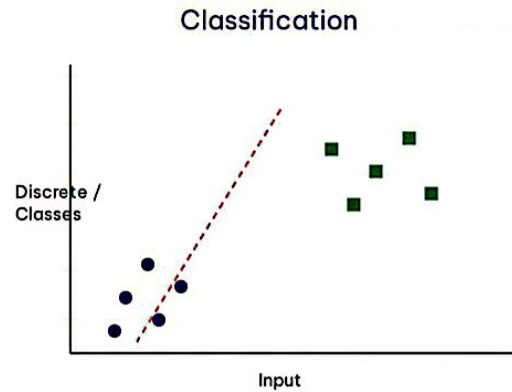
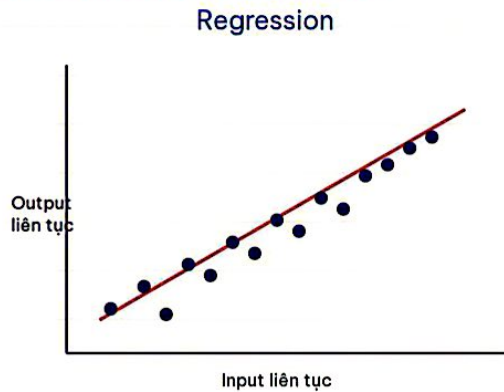
7. Regression

Regression là một loại supervised learning trong đó model dự đoán giá trị số liên tục (continuous numerical values).

Nó tìm ra mối quan hệ giữa input feature và một output liên tục.

Ví dụ: Dự đoán giá nhà, nhiệt độ, lương, doanh số, v.v.

Regression so với Classification



Các Thuật Toán Regression Phổ Biến

- Linear Regression
- Polynomial Regression
- Ridge Regression (L2 Regularization)
- Lasso Regression (L1 Regularization)
- Elastic Net Regression

Linear Regression (Đơn giản)

Tìm đường thẳng phù hợp nhất (best fit line):

$$y = mx + c$$

với, y = giá trị dự đoán

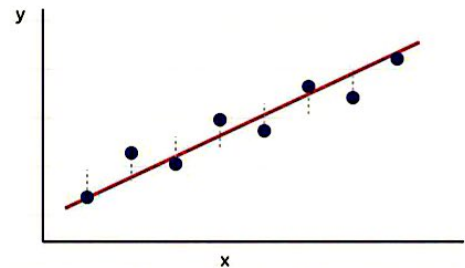
x = input feature

m = độ dốc (slope) (thay đổi của y theo x)

c = hệ số chặn (intercept) (y khi $x = 0$)

Cách Linear Regression Hoạt Động

- Cho dataset gồm các cặp (x, y) .
- Model tìm đường thẳng ($y = mx + c$) sao cho sai số nhỏ nhất.
- Sai số được đo bằng các metric như **Mean Squared Error**.
- Đường thẳng có sai số nhỏ nhất được xem là fit tốt nhất.



Khoảng cách theo chiều dọc từ điểm dữ liệu tới đường thẳng = sai số (residual)

Metric Đánh Giá (Regression)

1. MAE

Mean Absolute Error
Trung bình của trị tuyệt đối sai số.

2. MSE / RMSE

Phạt nặng hơn với sai số lớn. RMSE là căn bậc hai của MSE.

3. R^2 Score

Cho biết model giải thích được bao nhiêu variance trong dữ liệu (0 đến 1).

Regression được dùng khi câu trả lời là một con số, không phải một category.

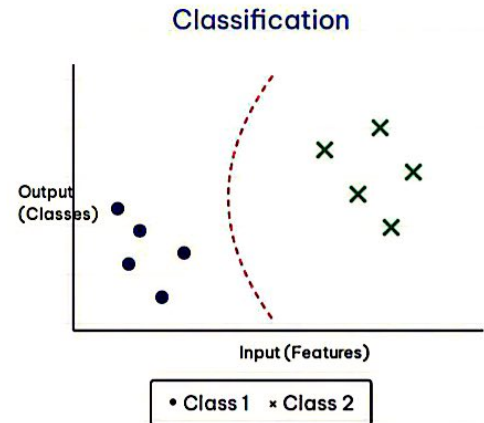
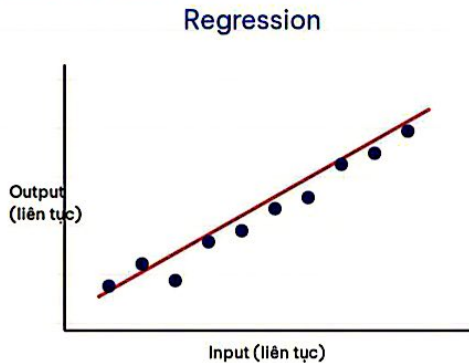
8. Classification

Classification là một loại supervised learning trong đó model dự đoán class hoặc category rời rạc (discrete classes).

Output là một class label, không phải giá trị liên tục.

Ví dụ: Phát hiện spam, phân loại email, chẩn đoán bệnh, nhận diện ảnh, v.v.

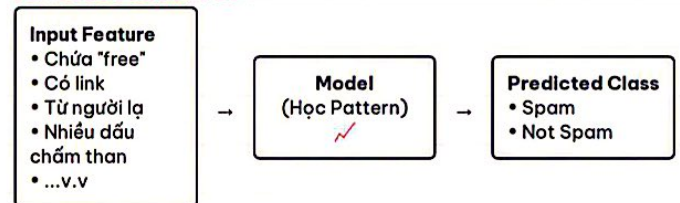
Regression so với Classification



Các Thuật Toán Classification Phổ Biến

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Decision Tree
- Random Forest
- Naive Bayes
- Gradient Boosting (XGBoost, LightGBM)

Classification Hoạt Động Thế Nào (VD: Phát Hiện Email Spam)



Model học pattern từ dữ liệu có nhãn và dự đoán class cho dữ liệu mới.

Metric Đánh Giá (Classification)

- Accuracy = $(TP + TN) / \text{Total}$
- Precision = $TP / (TP + FP)$
- Recall (Sensitivity) = $TP / (TP + FN)$
- F1 Score = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

TP = True Positive FP = False Positive

TN = True Negative FN = False Negative

Confusion Matrix

		Actual	
		Positive	Negative
Predicted	Positive	TP (True Positive)	FP (False Positive)
	Negative	FN (False Negative)	TN (True Negative)

Khi Nào Dùng Classification?

- Khi biến mục tiêu (target variable) là một category hoặc class.
- Khi câu trả lời là một label (ví dụ: Yes/No, Spam/Not Spam, Cat/Dog).

9. Clustering

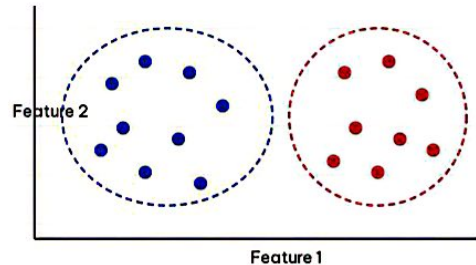
Clustering là một kỹ thuật **unsupervised learning**. Nó nhóm các điểm dữ liệu tương tự nhau lại dựa trên pattern, mà không sử dụng bất kỳ nhãn (label) định sẵn nào.

Ví dụ: Phân khúc khách hàng, gom nhóm tài liệu, phân đoạn ảnh, phát hiện bất thường.

Clustering Là Gì?

- Nhóm dữ liệu thành các cluster (hay segment).
- Dữ liệu trong cùng một cluster tương tự nhau hơn.
- Dữ liệu ở cluster khác nhau thì khác biệt hơn.

Ví Dụ Clustering (2 Cluster)

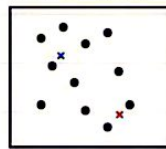


Các Thuật Toán Clustering Phổ Biến

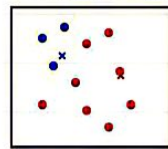
1. K-Means Clustering
2. Hierarchical Clustering
3. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
4. Gaussian Mixture Models (GMM)

K-Means Clustering (Ví Dụ)

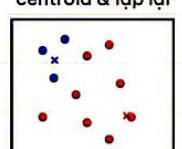
Bước 1: Chọn K (K=2)



Bước 2: Gán điểm



Bước 3: Cập nhật centroid & lặp lại



Lặp lại đến khi centroid không đổi.

Ứng Dụng

- Phân khúc khách hàng
- Hệ thống gợi ý
- Phân tích giỏ hàng (market basket)
- Gom nhóm tài liệu
- Phát hiện bất thường

Clustering Hoạt Động Thế Nào (Các Bước Chung)

1. Chọn số lượng cluster (K).
2. Khởi tạo centroid (ngẫu nhiên hoặc theo phương pháp).
3. Gán mỗi điểm dữ liệu vào centroid gần nhất.
4. Cập nhật centroid dựa trên các điểm đã gán.
5. Lặp lại bước 3 và 4 đến khi hội tụ.

Loại Dữ Liệu

- Dữ liệu số → Dùng distance measure (VD: Euclidean distance)
- Dữ liệu categorical → Dùng similarity measure (VD: Jaccard, Hamming)
- Dữ liệu hỗn hợp → Dùng hàm distance đặc biệt hoặc encoding

Distance Measure (Phổ Biến)

- Euclidean Distance: Cho dữ liệu số
- Manhattan Distance: Khoảng cách kiểu bàn cờ (city-block)
- Cosine Similarity: Cho dữ liệu chiều cao (high-dimensional)

Tóm Lại:

Clustering tìm ra các nhóm tự nhiên trong dữ liệu mà không cần nhãn. Nó giúp khám phá pattern và cấu trúc ẩn.

10. Các Thuật Toán ML Quan Trọng

Các thuật toán khác nhau được dùng cho **các loại bài toán khác nhau**. Chọn đúng thuật toán rất quan trọng để có hiệu năng model tốt. Dưới đây là một số thuật toán ML thường dùng nhất.

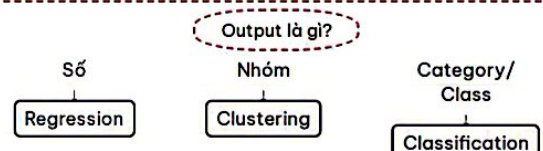
Thuật Toán	Loại	Use Case / Chức Năng	Điểm Chính
1. Linear Regression	Regression	- Dự đoán giá trị liên tục - Tìm đường thẳng phù hợp nhất	- Đơn giản và nhanh - Giả định quan hệ tuyến tính
2. Logistic Regression	Classification	- Dự đoán kết quả nhị phân - Xuất ra xác suất (0 đến 1)	- Dùng hàm sigmoid - Tốt cho binary classification
3. Decision Tree	Classification / Regression	- Chia dữ liệu dựa trên điều kiện - Tạo cấu trúc dạng cây	- Dễ diễn giải - Có thể overfit nếu cây quá sâu
4. Random Forest	Classification / Regression	- Ensemble của nhiều decision tree - Cải thiện độ chính xác & giảm overfitting	- Hoạt động tốt trên dataset lớn - Xử lý tốt dữ liệu phi tuyến
5. Support Vector Machine (SVM)	Classification / Regression	- Tìm ranh giới tốt nhất (hyperplane) - Hoạt động tốt trong không gian nhiều chiều	- Hiệu quả với dataset nhỏ - Kernel trick cho dữ liệu phi tuyến
6. K-Nearest Neighbors (KNN)	Classification / Regression	- Phân loại dựa trên láng giềng gần nhất - Đơn giản & instance-based learning	- Không có giai đoạn training - Chậm trên dataset lớn
7. Naive Bayes	Classification	- Bộ phân loại xác suất - Dựa trên định lý Bayes	- Nhanh và hiệu quả - Hoạt động tốt với dữ liệu văn bản
8. K-Means Clustering	Clustering	- Nhóm dữ liệu thành K cluster - Unsupervised learning	- Đơn giản & hiệu quả - Cần biết trước giá trị K

Tóm Tắt Theo Loại Thuật Toán

Thuật toán Regression (Dự đoán số)	→ Linear Regression, Decision Tree Regression, Random Forest Regression, SVR, v.v.
Thuật toán Classification (Dự đoán class)	→ Logistic Regression, Decision Tree, Random Forest, SVM, KNN, Naive Bayes, v.v.
Thuật toán Clustering (Tìm nhóm)	→ K-Means, Hierarchical Clustering, DBSCAN, Gaussian Mixture Models, v.v.

Ghi Nhớ:

- Hiểu rõ bài toán trước tiên.
- Kiểm tra loại output bạn cần.
- Bắt đầu đơn giản, sau đó thử model phức tạp hơn.
- Đánh giá và so sánh hiệu năng.



Tóm Lại:

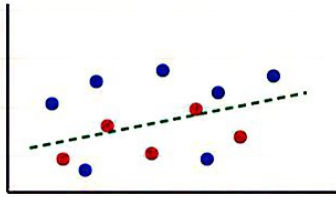
Không có thuật toán nào là tốt nhất cho mọi bài toán. Hiểu dữ liệu, thử nhiều thuật toán khác nhau, và chọn ra thuật toán cho kết quả tốt nhất.

11. Overfitting và Underfitting

Đây là những tình huống xảy ra khi model không thể generalize (khái quát hoá) tốt với dữ liệu mới, chưa từng thấy.

Chúng ta muốn một model hoạt động tốt trên cả **training data** và **dữ liệu chưa thấy (test)**.

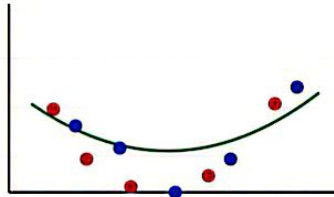
1. Underfitting
(High Bias)



- Model quá đơn giản để nắm bắt pattern.
- Training error cao
- Test error cao

Model bỏ lỡ xu hướng (quá nhiều bias)

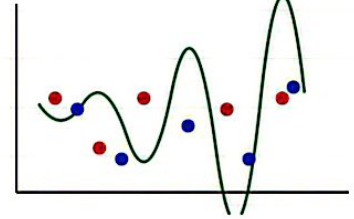
2. Good Fit
(Vừa Đúng)



- Model nắm bắt pattern tốt.
- Training error thấp
- Test error thấp

Model generalize tốt (hiệu năng tốt nhất)

3. Overfitting
(High Variance)



- Model quá phức tạp, fit cả nhiễu (noise).
- Training error thấp
- Test error cao

Model học cả noise (quá nhiều variance)

So Sánh

Khía Cạnh	Underfitting (High Bias)	Good Fit (Vừa Đúng)	Overfitting (High Variance)
Độ Phức Tạp Model	Quá đơn giản	Vừa đúng	Quá phức tạp
Training Error	Cao	Thấp	Rất thấp
Test/Validation Error	Cao	Thấp	Cao
Biểu Hiện	Bỏ lỡ pattern quan trọng	Nắm bắt pattern nền tảng	Fit cả nhiễu trong training data
Generalization	Kém	Tốt	Kém

Tại Sao Lại Xảy Ra?

- Underfitting → Model quá đơn giản, không học được pattern (High Bias).
- Overfitting → Model quá phức tạp, học cả noise lẫn pattern (High Variance).
- Good Fit → Cân bằng giữa Bias và Variance.

Cách Khắc Phục?

Khắc Phục Underfitting

- Dùng model phức tạp hơn
- Thêm feature
- Train nhiều epoch hơn
- Giảm regularization

Khắc Phục Overfitting

- Dùng model đơn giản hơn
- Thu thập thêm dữ liệu
- Thêm regularization (L1/L2)
- Dùng Dropout / Early Stopping

Điểm Mấu Chốt:

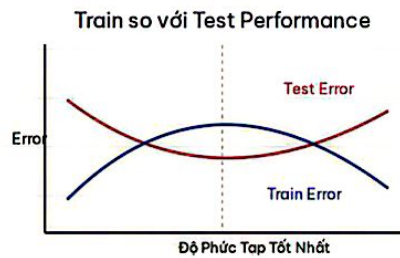
Mục tiêu là tìm điểm cân bằng nơi model học được pattern thật trong dữ liệu và hoạt động tốt trên dữ liệu mới, chưa từng thấy.

12. Đánh Giá Model

Model evaluation là quá trình đo lường mức độ hoạt động tốt của một model machine learning trên dữ liệu chưa từng thấy. Nó giúp ta chọn ra model tốt nhất và hiểu điểm mạnh, điểm yếu của nó.

Mục tiêu: Dự đoán tốt trên dữ liệu mới, thực tế.

- Đo lường hiệu năng model
- So sánh các model khác nhau
- Phát hiện overfitting hoặc underfitting
- Cải thiện và fine-tune model
- Xây dựng độ tin cậy cho dự đoán của model



Underfitting → Bias cao, cả hai error đều cao

Good Fit → Test error thấp

Overfitting → Train error thấp, test error cao

Metric Đánh Giá

Loại	Metric	Công Thức / Mô Tả	Phù Hợp Cho
Regression (Output liên tục)	MAE	Trung bình trị tuyệt đối sai số	Ổn định với outlier
	MSE	Trung bình bình phương sai số	Phạt nặng sai số lớn
	RMSE	Căn bậc hai của MSE	Cùng đơn vị với target
	R ² Score	$1 - \frac{SS_{res}}{SS_{tot}}$	Giải thích variance (0 đến 1)
Classification (Output nhị phân)	Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Class cân bằng
	Precision	$\frac{TP}{TP+FP}$	Giảm thiểu false positive
	Recall	$\frac{TP}{TP+FN}$	Giảm thiểu false negative
	F1 Score	$2 \times \frac{P \times R}{P+R}$	Khi class mất cân bằng
	ROC-AUC Score	Diện tích dưới đường ROC (0 đến 1)	Chất lượng phân loại tổng thể
Classification (Output đa lớp)	Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Multi-class cân bằng
	Macro Avg	Trung bình metric cho từng class	Xử lý class mất cân bằng
	Confusion Matrix	Bảng actual so với predicted class	Phân tích lỗi chi tiết

Confusion Matrix (Binary)

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Cross-Validation

- Chia dữ liệu thành k fold (VD: k=5)
 - Train trên k-1 fold, validate trên 1 fold
 - Lặp lại k lần
 - Lấy điểm trung bình
- Đánh giá đáng tin cậy hơn

Lưu Ý

- Đừng đánh giá trên training data
- Chỉ dùng test data một lần ở cuối
- Luôn kiểm tra overfitting
- Dùng metric phù hợp với bài toán

Tóm Lại:

Đánh giá tốt cho biết model của bạn generalize tốt như thế nào với dữ liệu mới. Chọn đúng metric, validate đúng cách, và luôn kiểm tra trên dữ liệu chưa thấy.

13. Feature Engineering

Feature Engineering là quá trình **tạo, biến đổi và chọn lọc** đúng feature từ dữ liệu thô để cải thiện hiệu năng model. Feature tốt → Học tốt hơn → Dự đoán tốt hơn.

Mục tiêu: Làm cho dữ liệu hữu ích hơn đối với model.

Tại Sao Lại Quan Trọng?

- Cải thiện độ chính xác của model
- Giảm thời gian huấn luyện
- Giúp model generalize tốt hơn
- Có thể giảm overfitting
- Đôi khi quan trọng hơn cả thuật toán!

Các Loại Feature Engineering

1. **Creation** = Tạo feature mới từ feature hiện có.
2. **Transformation** = Scale, encode, hoặc convert feature.
3. **Selection** = Giữ feature quan trọng, bỏ feature vô ích.
4. **Extraction** = Giảm chiều dữ liệu bằng kỹ thuật riêng.
5. **Interaction** = Kết hợp feature để nắm bắt quan hệ.

Dữ Liệu Thô so với Feature Đã Xử Lý

Age	Date of Birth	Salary
25	12-05-1999	40.000
30	23-11-1994	60.000
45	02-07-1979	90.000
...	...	...

Age	Kinh Nghiệm	Salary (Triệu)	Nhóm Tuổi
25	2	4.0	20-30
30	5	6.0	30-40
45	20	9.0	40+
...	...	...	...

Kỹ Thuật Thường Dùng

- Xử lý Missing Values
- Encode Dữ Liệu Categorical
- Scale Dữ Liệu Số
- Tạo Feature Mới
- Binning / Bucketing
- Feature Interaction
- Dimensionality Reduction (PCA)
- Feature Selection (Filter / Wrapper / Embedded)

1. Xử Lý Missing Values

- Xoá row/column có quá nhiều missing values.
- Impute (điền) missing values.
 - Mean/Median (số)
 - Mode (categorical)
 - Forward/Backward fill (time series)

2. Encode Dữ Liệu Categorical

- **Label Encoding** = Cho dữ liệu ordinal
- **One-Hot Encoding** = Cho dữ liệu nominal
- **Target Encoding** = Dùng trung bình target
- **Binary Encoding** = Tiết kiệm bộ nhớ

3. Scale Dữ Liệu Số

- **Standard Scaling (Z-score)**

$$x' = (x - \mu) / \sigma$$

- **Min-Max Scaling**

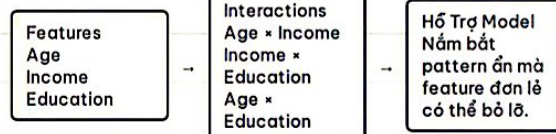
$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

4. Tạo Feature Mới (Ví Dụ)

Kỹ Thuật	Ví Dụ	Use Case
Date Features	Từ "2024-05-01" → Năm, Tháng, Ngày, Thứ	Time series, Seasonality
Aggregations	Tổng lượt mua, điểm TB mỗi user	User behavior, Transactions
Ratios	Spend / Income	Dữ liệu tài chính
Differences	Giá trị hiện tại - trước đó	Xu hướng, Thay đổi
Log / Power	log(Salary), sqrt(Price)	Dữ liệu bị lệch (skewed)

5. Feature Interaction

Kết hợp hai hoặc nhiều feature để nắm bắt mối quan hệ.



6. Phương Pháp Feature Selection

- **Filter:** Correlation, Chi-Square, Mutual Information
- **Wrapper:** Forward Selection, Backward Elimination
- **Embedded:** L1 (Lasso), Tree-based Importance

Mẹo Hay

- ✓ Hiểu sâu dữ liệu của bạn.
- ✓ Visualize trước & sau khi xử lý.
- ✓ Bắt đầu đơn giản, sau đó thêm độ phức tạp.
- ✓ Luôn validate tác động.

Tóm Lại

Feature tốt biến bài toán khó thành dễ đối với model.
Đầu tư thời gian vào feature engineering! ★

14. Cải Thiện Model

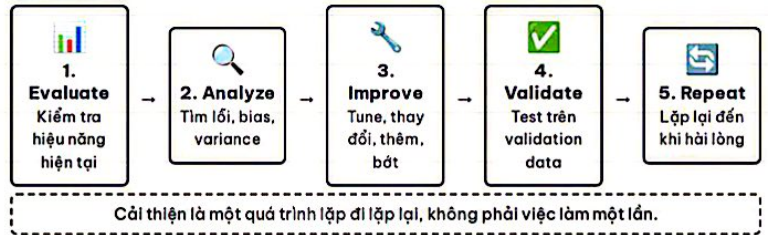
Model Improvement là quá trình nâng cao **hiệu năng**, **khả năng generalize** và **độ tin cậy** của model machine learning. Model tốt không được xây dựng trong một lần → chúng được cải thiện từng bước.

Mục tiêu: Có được model tốt nhất, hoạt động tốt trên dữ liệu chưa thấy.

Tại Sao Cần Cải Thiện Model?

- Tăng độ chính xác và giảm sai số
- Cải thiện khả năng generalize với dữ liệu mới
- Giảm overfitting / underfitting
- Giúp model mạnh mẽ (robust) hơn
- Tiết kiệm thời gian và compute khi production
- Tăng tác động kinh doanh và độ tin cậy

Chu Trình Cải Thiện Model



Các Cách Cải Thiện Model

Khu Vực	Cần Làm Gì	Giúp Ích Thế Nào	Ví Dụ / Kỹ Thuật
1. Dữ Liệu Nhiều & Tốt Hơn	Thu thập thêm dữ liệu Cải thiện chất lượng dữ liệu	Giảm overfitting Cải thiện generalization	Lấy thêm mẫu Loại bỏ nhiễu, sửa nhân
2. Feature Engineering	Tạo feature mới Chọn feature quan trọng	Nắm bắt pattern tốt hơn Giảm độ phức tạp	Polynomial features Feature selection (PCA, L1)
3. Hyperparameter Tuning	Tune tham số model Tìm cài đặt tối ưu	Hiệu năng tốt hơn Tránh đoán mò thủ công	Grid Search, Random Search Bayesian Optimization
4. Độ Phức Tạp Model	Điều chỉnh dung lượng model Dùng độ phức tạp phù hợp	Tránh underfitting/overfitting Cân bằng bias-variance tốt hơn	Đổi depth, số tree, C, k Regularization (L1, L2, Dropout)
5. Ensemble	Kết hợp nhiều model Dùng phương pháp ensemble	Dự đoán mạnh mẽ hơn Giảm variance/bias	Bagging, Boosting, Stacking Random Forest, XGBoost
6. Chọn Thuật Toán/Model	Thử nhiều thuật toán khác nhau Chọn cái phù hợp	Một số model fit dữ liệu tốt hơn Xử lý tốt loại bài toán	Linear → Tree → Boosting SVM → Neural Network

Chẩn Đoán Vấn Đề Trước Tiên

Underfitting (High Bias)

- Model quá đơn giản
- Giải pháp: Tăng độ phức tạp, thêm feature, train lâu hơn

Overfitting (High Variance)

- Model quá phức tạp
- Giải pháp: Thêm dữ liệu, regularization, dropout, early stopping

Sai Số Cao (High Error)

- Dữ liệu xấu / feature sai
- Giải pháp: Làm sạch dữ liệu, thiết kế feature tốt hơn

Kỹ Thuật Nên Thử

- **Cross-Validation** → Đảm bảo hiệu năng ổn định
- **Learning Curves** → Hiểu bias/variance
- **Regularization** → L1, L2, Dropout
- **Early Stopping** → Dừng khi không còn cải thiện
- **Data Augmentation** → Tạo dữ liệu mới từ dữ liệu có sẵn
- **Ensembling** → Kết hợp các model yếu thành model mạnh

Chiến Lược Cải Thiện (Thứ Tự Ưu Tiên)

1. Hiểu bài toán & metric
2. Cải thiện chất lượng dữ liệu
3. Thực hiện feature engineering
4. Tune hyperparameter
5. Điều chỉnh độ phức tạp model
6. Thử ensemble hoặc model nâng cao
7. Validate và test trên dữ liệu chưa thấy

- ✓ Cải thiện nhỏ ở mỗi bước dẫn đến kết quả lớn.
- ✓ Luôn validate thay đổi trước khi qua bước tiếp theo.

Tóm Lại:

Một model tốt không phải nhờ may mắn → đó là nhờ cải tiến liên tục. Hãy tiếp tục đánh giá, phân tích, và lặp lại cho đến khi đạt kết quả tốt nhất!

15. ML Model dưới dạng API

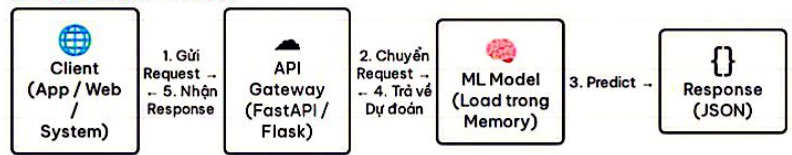
Sau khi model được huấn luyện và đánh giá, ta deploy nó dưới dạng một API để các ứng dụng có thể gửi dữ liệu và nhận dự đoán theo **real-time**. Điều này giúp model có thể được sử dụng bởi hệ thống, app, hoặc người dùng khác.

Mục tiêu: Làm cho ML model dễ truy cập, scalable, và đáng tin cậy.

Tại Sao Expose Model Thành API?

- Dự đoán real-time cho người dùng
- Dễ tích hợp với app web/mobile
- Scalable – xử lý nhiều request
- Tái sử dụng trên nhiều nền tảng
- Tách biệt logic ML và logic ứng dụng

Kiến Trúc Tổng Quan



API Endpoint (Ví Du)

Endpoint	Method	Mô Tả	Request Body	Response
/predict	POST	Lấy dự đoán từ model	{"features": [x1, x2, x3, ...]}	{"prediction": "Yes", "probability": 0.87}
/health	GET	Kiểm tra API có đang chạy	–	{"status": "OK"}
/info	GET	Lấy thông tin model	–	{"model": "RandomForest", "version": "1.0", "features": 20}

Luồng Ví Du (Prediction)

1. Client gửi dữ liệu đầu vào (feature)
2. API nhận request tại /predict
3. API validate dữ liệu đầu vào
4. Dữ liệu được đưa vào model đã load
5. Model thực hiện dự đoán
6. API trả về dự đoán dưới dạng JSON response

Mẫu API Request & Response

Request (POST /predict)

```
{
  "features": [5.1, 3.5, 1.4, 0.2]
}
```

Response

```
{
  "prediction": "setosa",
  "probability": 0.98
}
```

Công Cụ & Công Nghệ

Framework	FastAPI, Flask
Model Serialization	Joblib, pickle, ONNX
Data Validation	Pydantic (FastAPI), Cerberus
Deployment	Docker, AWS (EC2, Lambda), Render, Heroku
Testing	Postman, curl, pytest

Các Bước Deploy Model Thành API

1. Train và save model
2. Build API bằng framework (FastAPI / Flask)
3. Load model vào memory
4. Tạo endpoint (VD: /predict)
5. Test API ở local
6. Deploy lên server / cloud
7. Giám sát và bảo trì

Lợi Ích:

Dự đoán real-time • Scalable và đáng tin cậy • Dễ tích hợp • Dễ bảo trì và giám sát

16. Machine Learning trong Production

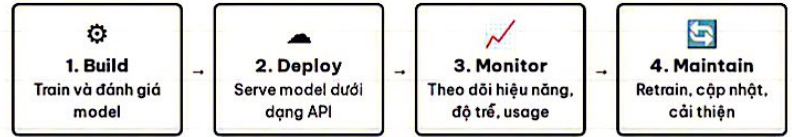
Xây dựng model chỉ mới là khởi đầu. Tác động thực sự đến khi model được **deploy**, sử dụng, giám sát, và cải thiện trong thế giới thực.

Mục tiêu: Cung cấp dự đoán đáng tin cậy ở quy mô lớn và tiếp tục cải thiện theo thời gian.

Tại Sao Đưa ML Vào Production?

- Giải quyết vấn đề thực tế
- Tự động hoá quyết định và tiết kiệm thời gian
- Tạo ra giá trị kinh doanh
- Liên tục học từ dữ liệu mới

Vòng Đời ML Production



Thành Phần Chính

Thành Phần	Mô Tả	Ví Dụ / Công Cụ
Data Pipeline	Thu thập, làm sạch, chuẩn bị dữ liệu liên tục.	Airflow, Kafka, AWS Glue
Model Serving	Expose model như một endpoint để dự đoán.	FastAPI, Flask, TensorFlow Serving, TorchServe
Monitoring	Theo dõi hiệu năng model, data drift, system health.	Prometheus, Grafana, Evidently, WhyLabs
Logging	Log request, response, và lỗi để debug.	ELK Stack, CloudWatch
Versioning	Theo dõi version model và dữ liệu để đảm bảo reproducibility.	MLflow, DVC, Git
Retraining Pipeline	Tự động hoá việc retrain với dữ liệu mới và validation.	Airflow, Kubeflow, SageMaker

Thách Thức Trong Production

- Data drift (dữ liệu thay đổi theo thời gian)
- Concept drift (pattern thay đổi)
- Độ trễ và khả năng mở rộng
- Model xuống cấp (degradation)
- Bảo mật và quyền riêng tư
- Chi phí hạ tầng

Thực Hành Tốt Nhất

- Bắt đầu đơn giản và lặp lại
- Giám sát mọi thứ quan trọng
- Đặt alert cho lỗi và drift
- Tự động hoá việc retrain
- Tài liệu hoá và version mọi thứ
- Luôn có sự tham gia của con người

Giám Sát Cái Gì?

Chất Lượng Dữ Liệu

→ Missing values, thay đổi phân phối

Hiệu Năng Model

→ Accuracy, precision, recall, v.v.

Phân Phối Dự Đoán

→ Khoảng output, confidence score

System Health

→ Độ trễ, tỷ lệ lỗi, uptime

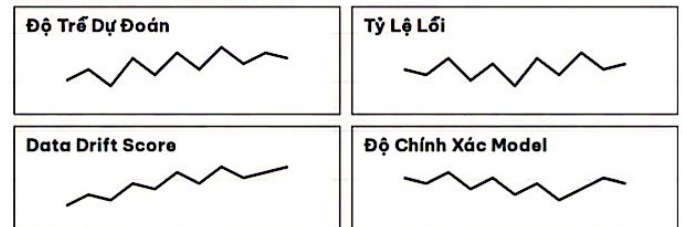
Usage Metrics

→ Số lượng request, user, khu vực

Tùy Chọn Deploy

Tùy Chọn	Mô Tả
On-Premise Server	Deploy trên server công ty.
Cloud (VM / Container)	Deploy dùng Docker trên AWS, GCP, Azure.
Serverless	Dùng AWS Lambda, GCP Cloud Run, v.v.
Managed ML Services	SageMaker, Vertex AI, Azure ML.

Dashboard Giám Sát Mẫu



Ghi Nhớ

- Model không bao giờ "hoàn thành".
- Production là một quá trình liên tục.
- Giám sát → Học hỏi → Cải thiện → Lặp lại.